

Chapter 12 — Data lineage tracking

Versioning records what changed; lineage records how data moved and transformed

Learning objectives

- By the end of this part

Definition and importance

- Data lineage tracks where data came from, how it was transformed
- Origin, transforms and aggregations, and downstream dependents
- Chapter 8 covers provenance as documentation for individual datasets
- Lakes, and engines

Compliance and debugging

- Regulated industries need lineage evidence for regimes such as GDPR and HIPAA
- Large platforms also use lineage to debug broken dashboards by walking upstream transforms
- Operational gains include proving which jobs touched regulated tables and coordinating

Example 12.19 — Lineage Metadata on Schema and Steps

- Example 12.19 — hands-on module
- Metadata is central to lineage: it describes source, format, transformations, and usage
- Example 12.19 lists fields that feed lineage views
- Lineage catalogs record schema versions, transform job IDs
- Explore the chapter example module
- View files: ``modules/chapter12/example19/``

Example 12.20 — Automated Lineage Capture on Model Runs

- Example 12.20 — hands-on module
- Automated tracking systems monitor pipelines and capture lineage without manual input
- Example 12.20 captures lineage when a training job starts
- Explore the chapter example module
- View files: ``modules/chapter12/example20/``

Tools and Example 12.21

- Example 12.21 — hands-on module
- Apache Atlas is an open-source metadata and governance tool for lineage, cataloging
- Amundsen, developed at Lyft, emphasizes data discovery and integrates with Airflow and dbt
- Example 12.21 uses lineage graphs to find a bad filter step
- Explore the chapter example module
- View files: ``modules/chapter12/example21/``

Example 12.22 — Cross-Platform Lineage Gaps

- Example 12.22 — hands-on module
- The distinctive platform-scale challenge is cross-system coverage
- Organizations mix cloud storage, Spark, Hadoop, Kafka, and machine-learning platforms
- Example 12.22 shows lineage breaking across boundaries
- Explore the chapter example module
- View files: ``modules/chapter12/example22/``

Takeaways

- Lineage answers origin, transform, and dependency questions across pipelines
- Metadata and automated capture keep records current
- Visualization helps debug empty partitions and broken dashboards
- Cross-platform gaps remain the hard problem when connectors stop at system boundaries
- Distinguish Chapter 8 per-dataset provenance from Chapter 12 platform catalogs

Next

- Complete the quiz for this part
- The next part applies storage, streaming, and governance patterns in short case studies